Final Project: Design a Long-Running Agent Workflow for a Finance Task

This is an individual project. Choose a substantial research or analytical activity and design an end-to-end agent workflow for it. The workflow should connect several stages, retain state, support repeated evaluation, and recover from interruptions. A single prompt or report update is not enough.

1. Specify a substantial, recurring task

Choose an activity from your work or study. **Buy-side quantitative strategy research** could cover hypothesis development, data preparation, backtesting, robustness checks, portfolio construction, and ongoing monitoring. **Sell-side company or industry coverage** could involve source collection, financial modeling, valuation, report production, and continuing updates. Other tasks are welcome if their scope is comparable, even if their connection to finance is limited. Identify the intended user and describe the current process. Set out the stages, dependencies, recurring triggers, inputs, permitted sources, and information cutoffs. Specify the work products and acceptance criteria. Explain why the task needs a persistent workflow rather than a one-shot script.

2. Design the architecture using the lecture concepts

Provide an annotated workflow diagram and a short design note explaining:

- **Roles and handoffs:** Specify each agent's responsibilities, tools, reusable skills or instructions, and input/output artifacts. Explain the division of work and use deterministic code for calculations where appropriate. Distinguish agent decisions from execution control by the harness/controller.
- **Long-running operation:** Describe persistent state and checkpoints, dependencies, and scheduling or new-input triggers. Specify retries and rules for stopping or escalating problems. Explain how interrupted runs resume without repeating completed actions. Set limits on time, cost, and tool permissions.
- **Verification and improvement:** Describe checks independent of the producing agent, human approval before consequential actions, and a bounded revision loop. Explain how you would test a proposed change against a fixed baseline, accept or reject it, record its version, and roll it back. For time-sensitive tasks, prevent future-information leakage.

3. Submit one ZIP with complete Git history

Include the design note and diagram, prompts, skills, configuration, code if implemented, shareable example inputs and outputs, and evidence of verification and recovery. In the root `README.md`, give your identification, task summary, **repository URL** (if applicable), and a guide to the files. Provide setup and run instructions or a walkthrough, with any relevant dependencies. Briefly explain your own contribution to the project. Make meaningful commits throughout development, not just at the end.

Submit one ZIP of your **existing local project folder**, including all deliverables and the **complete hidden `.git/` directory**. Commit your final changes before compressing the folder. The repository must contain the full development history, not a shallow copy. Include hidden folders in the ZIP and name it `studentID-final-project.zip`.

A host's "Download ZIP" option and `git archive` omit Git history, so neither meets this requirement. The files and history must be reviewable without access to the hosted repository. Keep credentials, private client data, and other secrets out of both the submitted files and the commit history.